



Cloud Management Mechanisms

Resource, Architecture, Security & Governance

RAVI SAJJANAR

Senior Cloud Developer
Hewlett Packard Enterprise

Cloud Management Mechanisms

Session 1:

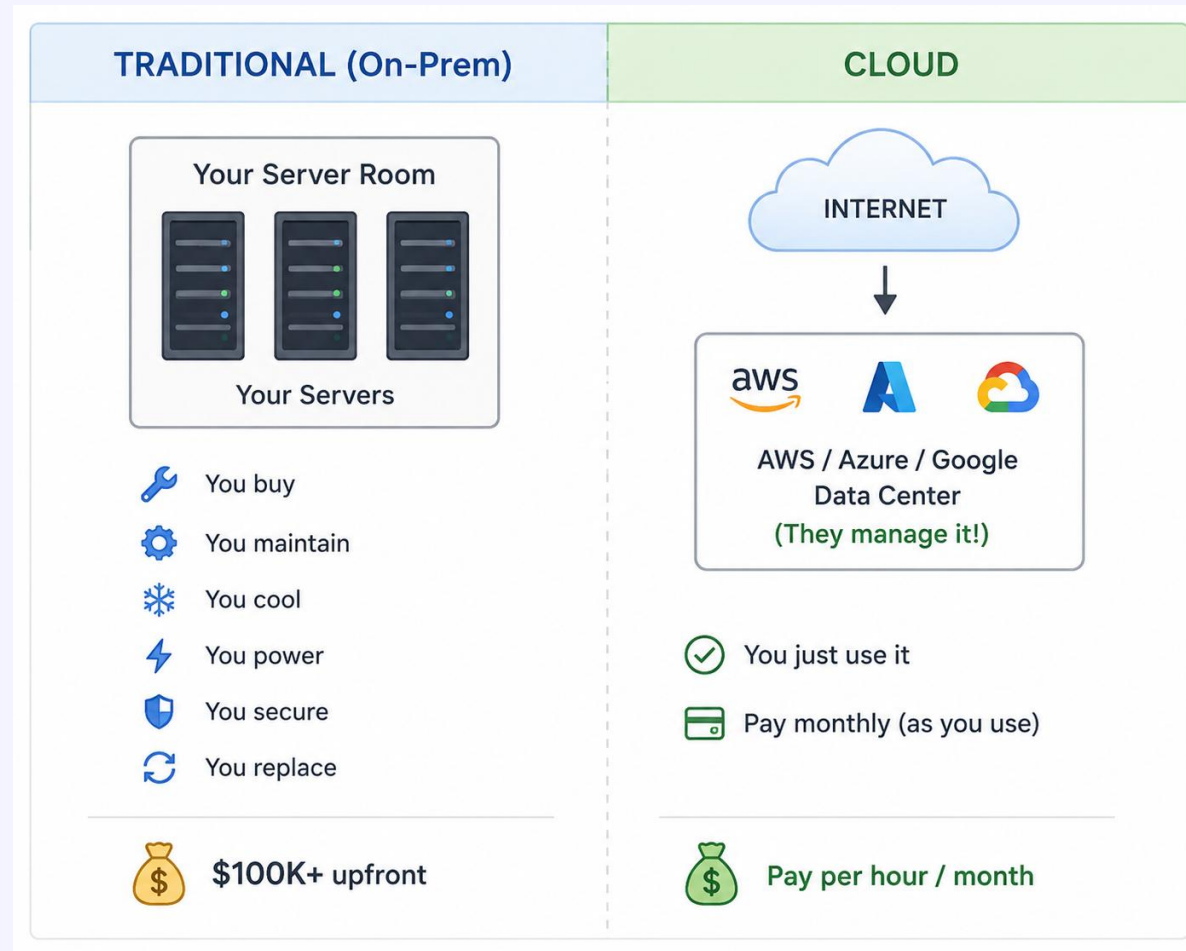
1. Cloud Computing Fundamentals
2. Cloud Computing Architectures
 - Workload Distribution
 - Elastic Resource Capacity
 - Multi Cloud
 - Hypervisor Clustering
 - Cloud Balancing
3. Specialized Cloud Architectures:
 - Edge Computing
 - Fog Computing
 - Meta Cloud
 - Federated Cloud

Session 2:

1. Cloud Computing Security
 - Threat Agents
 - Common Threats
2. Cybersecurity Access-Oriented Mechanisms
 - Cloud-Based Security Groups
 - Hardened Virtual Server Image
 - Identity and Access Management (IAM) System
3. Cybersecurity Data-Oriented Mechanisms:
 - Data Loss Prevention (DLP) System
 - Trusted Platform Module (TPM)
4. Cloud Delivery Model Considerations
 - Case Study on Cloud Provider and Consumer Perspective.

1.1 Cloud Computing Fundamentals

Cloud computing = On-demand delivery of compute, storage, and networking resources over the internet with pay-per-use pricing.



1.1 Cloud Computing Fundamentals

Cloud computing = On-demand delivery of compute, storage, and networking resources over the internet with pay-per-use pricing.

Cloud vs On-Premises Comparison

Aspect	On-Premises	Cloud
CapEx	High (buy hardware)	Low (rent resources)
Scaling Time	Weeks/months	Minutes
Maintenance	Your responsibility	Provider managed
Global Deployment	Build data centers	Already available
Disaster Recovery	Complex, expensive	Built-in options

1.1 Cloud Computing Fundamentals

Service Models

Model	Examples	You Manage
SaaS	Gmail, Zoom, Slack, Netflix, Canva, Figma, Google Docs, Spotify	Just use the app (zero setup!)
PaaS	Vercel, Netlify, Heroku, Railway, Google App Engine	Deploy your code (no server config)
IaaS	AWS EC2, Azure VMs, DigitalOcean, Google Compute Engine, Linode	Full VM control (OS, networking, etc.)

Major Providers (2026)

AWS (~31%): Most services, largest market share

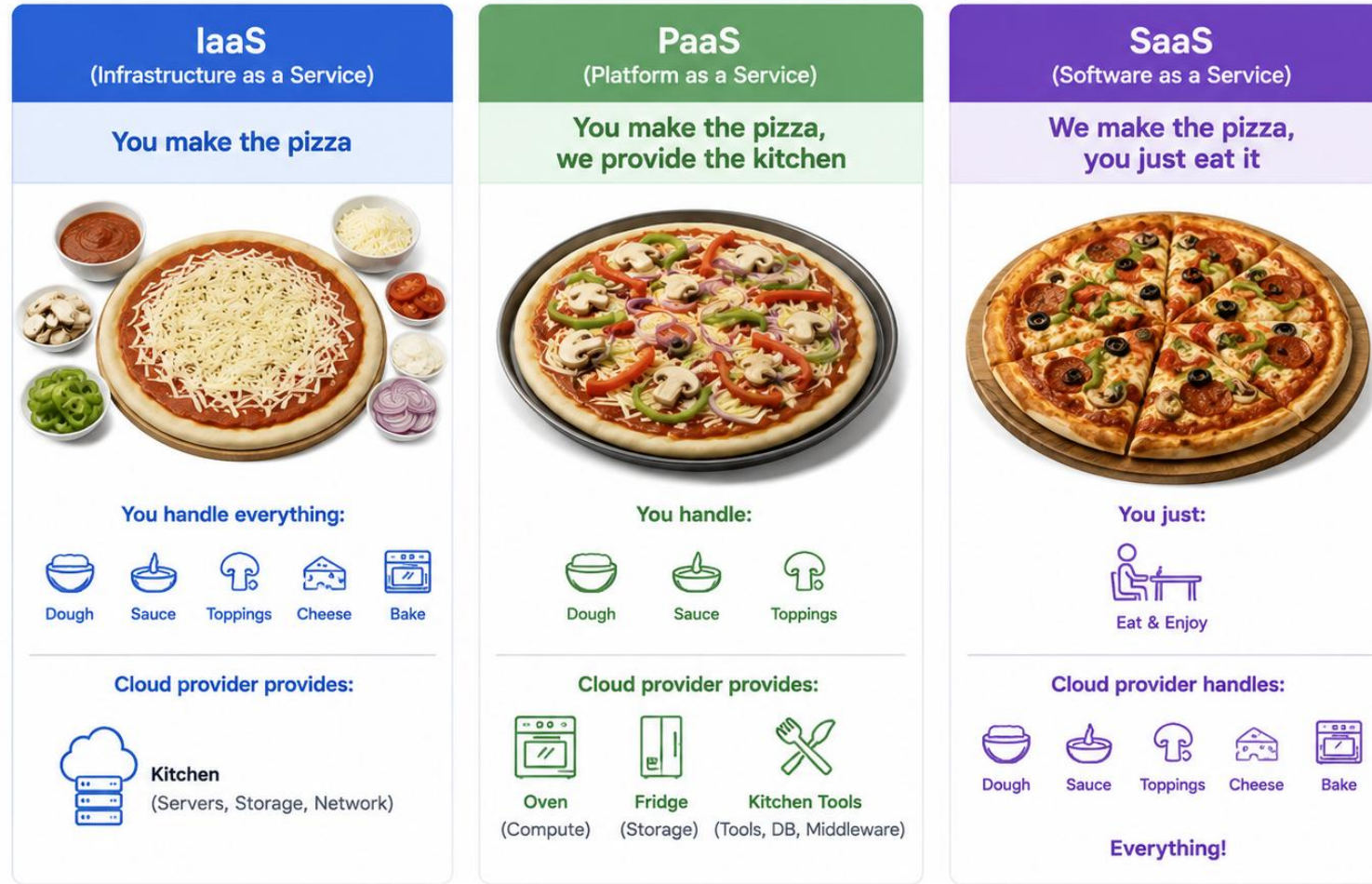
Azure (~25%): Enterprise integration, Microsoft ecosystem

GCP (~12%): AI/ML, Kubernetes-native

1.1 Cloud Computing Fundamentals

Cloud Service Models Analogy: Pizza

Different cloud service models = Different levels of management



Key Takeaway: IaaS = You rent the kitchen | PaaS = You cook in our kitchen | SaaS = We serve, you enjoy

Why Should YOU Care About Cloud?



CAREER IMPACT



SALARY BOOST:

- Cloud skills = **20–30%** higher salary
- AWS certification = avg **\$25K** more per year



JOB DEMAND:

- 2026: **4.5 million** cloud jobs unfilled globally
- Every company is moving to cloud



ROLES THAT NEED THIS:

- Software Developer
- System Administrator
- Data Engineer
- DevOps Engineer
- Solutions Architect
- Security Engineer



“If you're in IT and don't know cloud, you're falling behind.”

1.2 Cloud Management Mechanisms Overview



DEFINITION

Cloud Management Mechanisms are the tools, technologies, and architectural patterns used to **GOVERN**, **OPERATE**, and **OPTIMIZE** cloud computing resources.



Think of them as the “operating system” for managing the cloud:



How do we distribute work across servers?



Load Balancing



How do we grow/shrink resources automatically?



Auto-Scaling



How do we manage multiple cloud providers?



Multi-Cloud



How do we ensure high availability?



Clustering



How do we secure cloud resources?



Security Groups



How do we monitor and control costs?



Cloud Governance



In Short: Cloud Management Mechanisms = techniques to manage rented cloud resources efficiently, securely, and cost-effectively.

1.2 Cloud Management Mechanisms Overview

The **9 core mechanisms** for managing cloud resources:

Category	Mechanisms	Purpose
Resource Management	Workload Distribution, Elastic Capacity, Hypervisor Clustering	How to use cloud resources efficiently
Architecture Patterns	Multi-Cloud, Cloud Balancing, Metacloud, Federated	How to structure your cloud deployment
Edge Processing	Edge Computing, Fog Computing	How to control and audit your cloud Low-latency processing

1.3 Workload Distribution (Load Balancing)

Concept:

Distributes incoming traffic across multiple backend servers to prevent overload and ensure high availability.

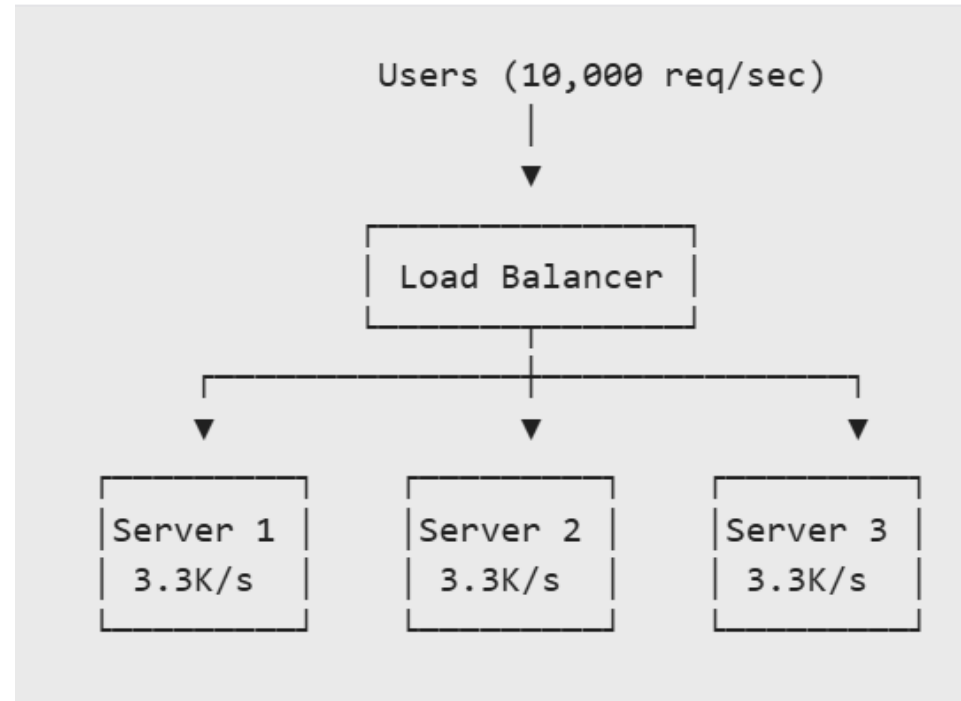
Real-World Analogy: Restaurant waiters

Imagine a busy restaurant with 5 chefs:

- Without waiters: All customers crowd one chef → long wait, chef burns out
- With waiters: Distributes orders evenly → faster service, happy customers

Load Balancer = Digital waiters for your servers

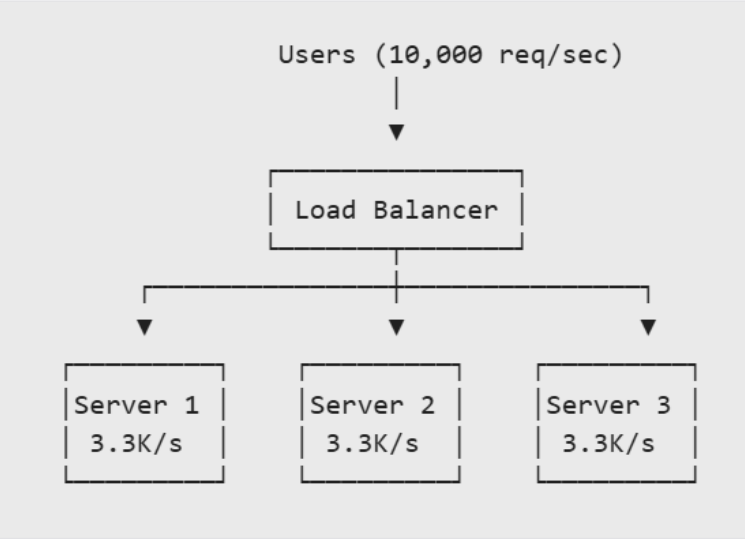
1.3 Workload Distribution (Load Balancing)



Application Examples

App	How Load Balancing Helps
Flipkart Big Billion Days	10M users hit site simultaneously → distributed across 1000s of servers
IRCTC Tatkal	10 AM rush → requests spread to prevent crash
WhatsApp	2B users → messages routed to nearest healthy server
YouTube	Video requests distributed based on viewer location

1.3 Workload Distribution (Load Balancing)



Load Balancing Algorithms

Algorithm	Description	Use Case
Round Robin	Sequential distribution	Equal server capacity
Weighted Round Robin	Based on server capacity	Heterogeneous servers
Least Connections	Routes to least busy server	Long-lived connections
IP Hash	Same client → same server	Session persistence
Least Response Time	Routes to fastest server	Performance-critical apps

1.3 Workload Distribution (Load Balancing)

AWS Implementation

Application Load Balancer (ALB) - Layer 7, HTTP/HTTPS routing

Create ALB using AWS CLI

```
aws elbv2 create-load-balancer \  
  --name my-alb \  
  --subnets subnet-12345 subnet-67890 \  
  --security-groups sg-12345
```

Create Target Group

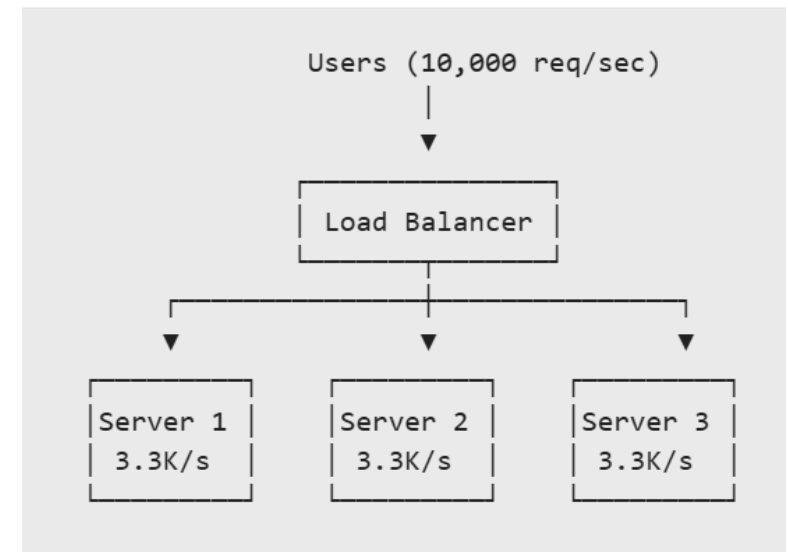
```
aws elbv2 create-target-group \  
  --name my-targets \  
  --protocol HTTP \  
  --port 80 \  
  --vpc-id vpc-12345 \  
  --health-check-path /health
```

Register EC2 instances

```
aws elbv2 register-targets \  
  --target-group-arn arn:aws:elasticloadbalancing:... \  
  --targets Id=i-instance1 Id=i-instance2
```

Health Checks Configuration:

- Path: `/health` or `/`
- Interval: 30 seconds
- Healthy threshold: 2 consecutive successes
- Unhealthy threshold: 2 consecutive failures



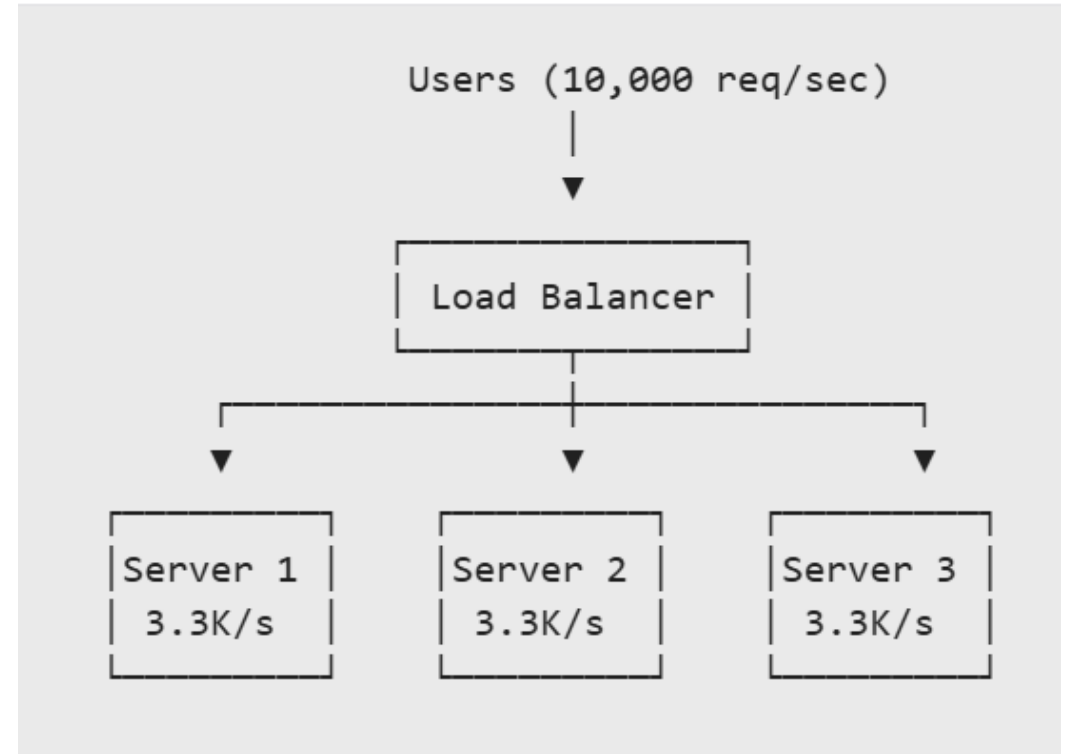
1.3 Workload Distribution (Load Balancing)

Real-World Example: Netflix

280M users globally

Uses AWS ELB with thousands of backend instances

Geographic routing via Route 53 + regional ALBs



1.4 Elastic Resource Capacity (Auto-Scaling)

Concept

Automatically adjusts compute capacity based on demand. Scales OUT (add instances) or IN (remove instances).

Real-World Analogy: Uber Surge Pricing

```
Uber during normal hours: 100 drivers active  
Uber on New Year's Eve: 500 drivers active (auto-scaled!)  
Uber at 3 AM: 20 drivers active (scaled down)
```

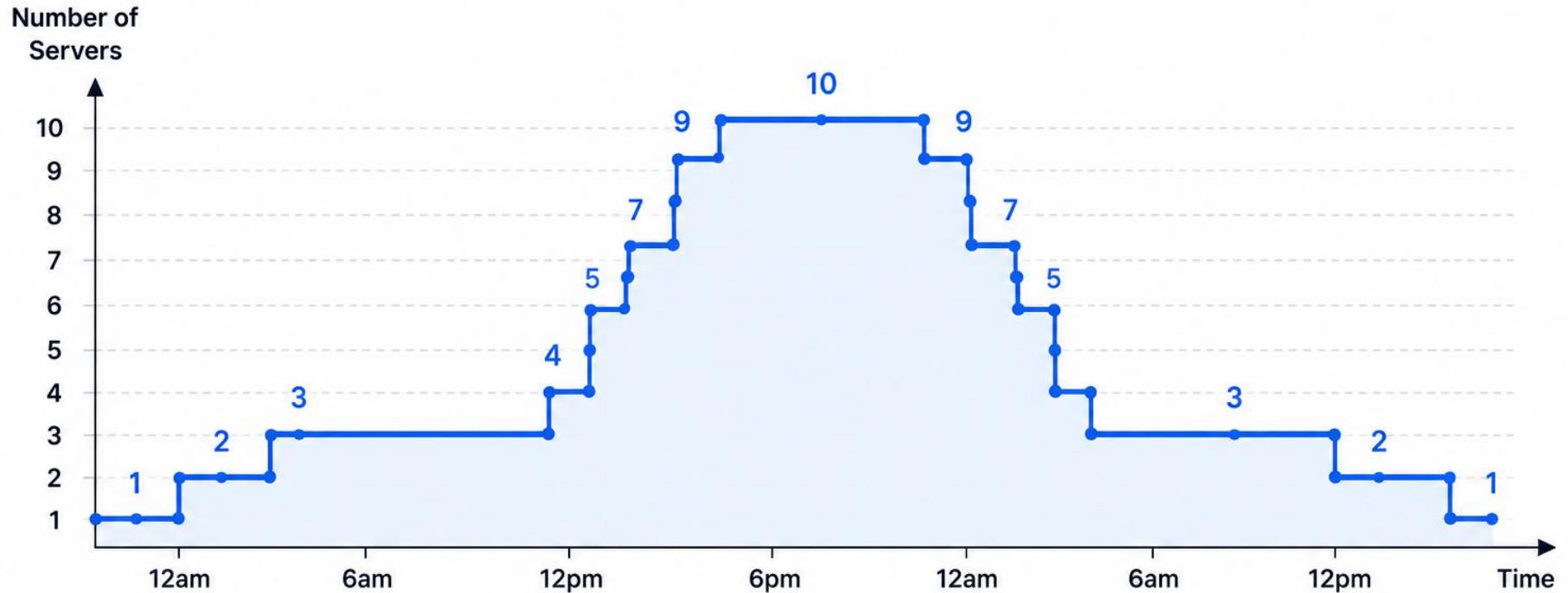
```
You don't pay 500 drivers 24/7 – only when needed.  
Same with cloud servers!
```

1.4 Elastic Resource Capacity (Auto-Scaling)



Auto-Scaling Timeline Graph

Elastic Resource Capacity (Auto-Scaling)



Normal Traffic



Traffic Rises



Peak Traffic



Traffic Drops



Normal Traffic



RESULT: Pay for 10 servers only during peak, not 24/7!

1.4 Elastic Resource Capacity (Auto-Scaling)

Application Examples

App	Scaling Scenario
Hotstar IPL Final	2 servers → 2000 servers in 10 minutes
Zomato lunch hour	12-2 PM: 5x servers, scales down by 4 PM
E-commerce flash sale	Pre-scheduled scale-up before sale starts
SaaS product	Weekdays busy, weekends quiet → auto-adjusts

1.4 Elastic Resource Capacity (Auto-Scaling)

Scaling Types

Type	Description	Example
Horizontal	Add/remove instances	2 servers → 10 servers
Vertical	Resize instance	t2.micro → t2.large

Best Practice: Prefer horizontal scaling (no single point of failure, unlimited capacity).

Scaling Policies

```
Target Tracking: Maintain CPU at 70%  
Step Scaling: +2 instances when CPU > 80%  
Scheduled: Scale to 10 instances at 9 AM daily
```

1.4 Elastic Resource Capacity (Auto-Scaling)

Vertical vs Horizontal Scaling

TWO WAYS TO SCALE



VERTICAL (Scale UP)
"Bigger Server"

Before:



After:



- ✓ Simple
- ✗ Has limits
- ✗ Downtime to upgrade



HORIZONTAL (Scale OUT)
"More Servers"

Before:



After:



- ✓ No single point of failure
- ✓ Unlimited scaling
- ✗ More complex



CLOUD BEST PRACTICE: Horizontal scaling (add more servers)

1.4 Elastic Resource Capacity (Auto-Scaling)

AWS Auto Scaling Configuration

Create Launch Template

```
aws ec2 create-launch-template \  
  --launch-template-name my-template \  
  --launch-template-data '{  
    "ImageId": "ami-12345",  
    "InstanceType": "t3.micro",  
    "SecurityGroupIds": ["sg-12345"]  
  }'
```

Create Auto Scaling Group

```
aws autoscaling create-auto-scaling-group \  
  --auto-scaling-group-name my-asg \  
  --launch-template LaunchTemplateName=my-template \  
  --min-size 2 \  
  --max-size 10 \  
  --desired-capacity 2 \  
  --vpc-zone-identifier "subnet-1,subnet-2"
```

Create Target Tracking Policy

```
aws autoscaling put-scaling-policy \  
  --auto-scaling-group-name my-asg \  
  --policy-name cpu-target-tracking \  
  --policy-type TargetTrackingScaling \  
  --target-tracking-configuration '{  
    "PredefinedMetricSpecification": {  
      "PredefinedMetricType": "ASGAverageCPUUtilization"  
    },  
    "TargetValue": 70.0  
  }'
```


1.4 Elastic Resource Capacity (Auto-Scaling)

Terraform Example

```
resource "aws_autoscaling_group" "web" {
  name                = "web-asg"
  vpc_zone_identifier = var.subnet_ids
  min_size            = 2
  max_size            = 10
  desired_capacity     = 2

  launch_template {
    id      = aws_launch_template.web.id
    version = "$Latest"
  }

  target_group_arns = [aws_lb_target_group.web.arn]

  tag {
    key      = "Name"
    value    = "web-server"
    propagate_at_launch = true
  }
}

resource "aws_autoscaling_policy" "cpu" {
  name                = "cpu-policy"
  autoscaling_group_name = aws_autoscaling_group.web.name
  policy_type          = "TargetTrackingScaling"

  target_tracking_configuration {
    predefined_metric_specification {
      predefined_metric_type = "ASGAverageCPUUtilization"
    }
    target_value = 70.0
  }
}
```

Cost Impact

- **Without auto-scaling:**
Pay for peak capacity 24/7
- **With auto-scaling:**
Pay for actual usage
- **Savings:**
40-70% for variable workloads

1.5 Multi-Cloud Architecture

Concept

Using multiple cloud providers simultaneously to avoid vendor lock-in, leverage provider strengths, and improve resilience.

Real-World Analogy: Investment Portfolio

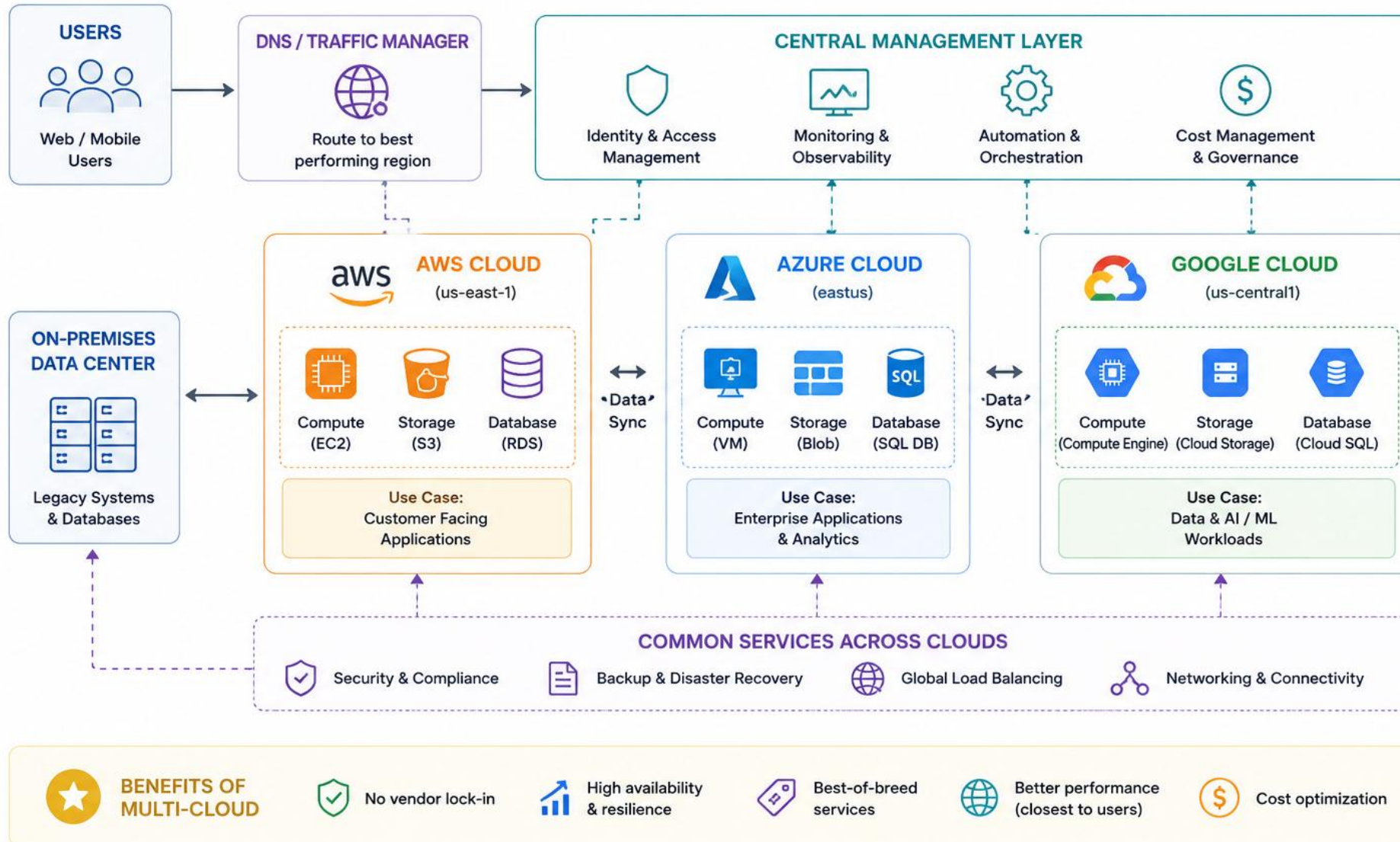
Don't put all your money in one stock!

- Stocks (AWS) + Bonds (Azure) + Gold (GCP) = Diversified portfolio
- If one crashes, others keep you stable

Same logic: Don't put all workloads on one cloud provider.

1.5 Multi-Cloud Architecture

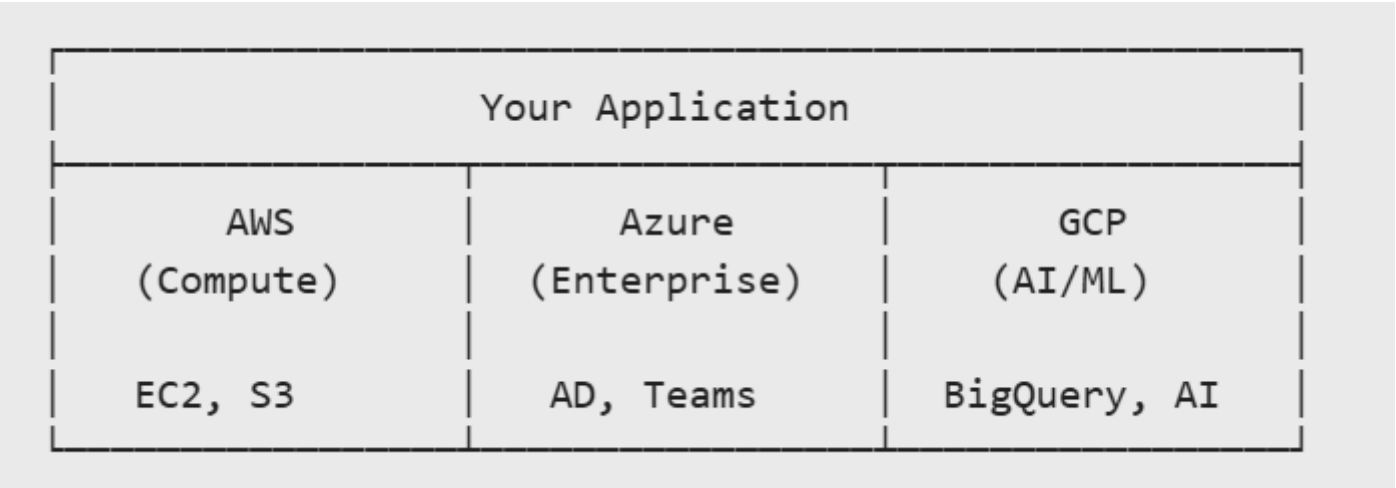
Use multiple cloud providers for flexibility, resilience, and optimal services



1.5 Multi-Cloud Architecture

Application Examples

Company	Multi-Cloud Strategy
Twitter/X	AWS (compute) + GCP (data analytics)
Spotify	GCP (ML/data) + AWS (some services)
Adobe	AWS + Azure for redundancy
Banks in India	RBI mandates: No single cloud dependency



1.5 Multi-Cloud Architecture

When to Use Multi-Cloud

Scenario	Benefit
Regulatory compliance	Data residency requirements
Vendor negotiation	Better pricing leverage
Best-of-breed services	Use each provider's strengths
Disaster recovery	Cross-provider failover

1.5 Multi-Cloud Architecture

Challenges

- Increased complexity (multiple CLIs, APIs, skills)
- Data transfer costs between clouds
- Inconsistent security policies
- Higher operational overhead

Implementation Tools

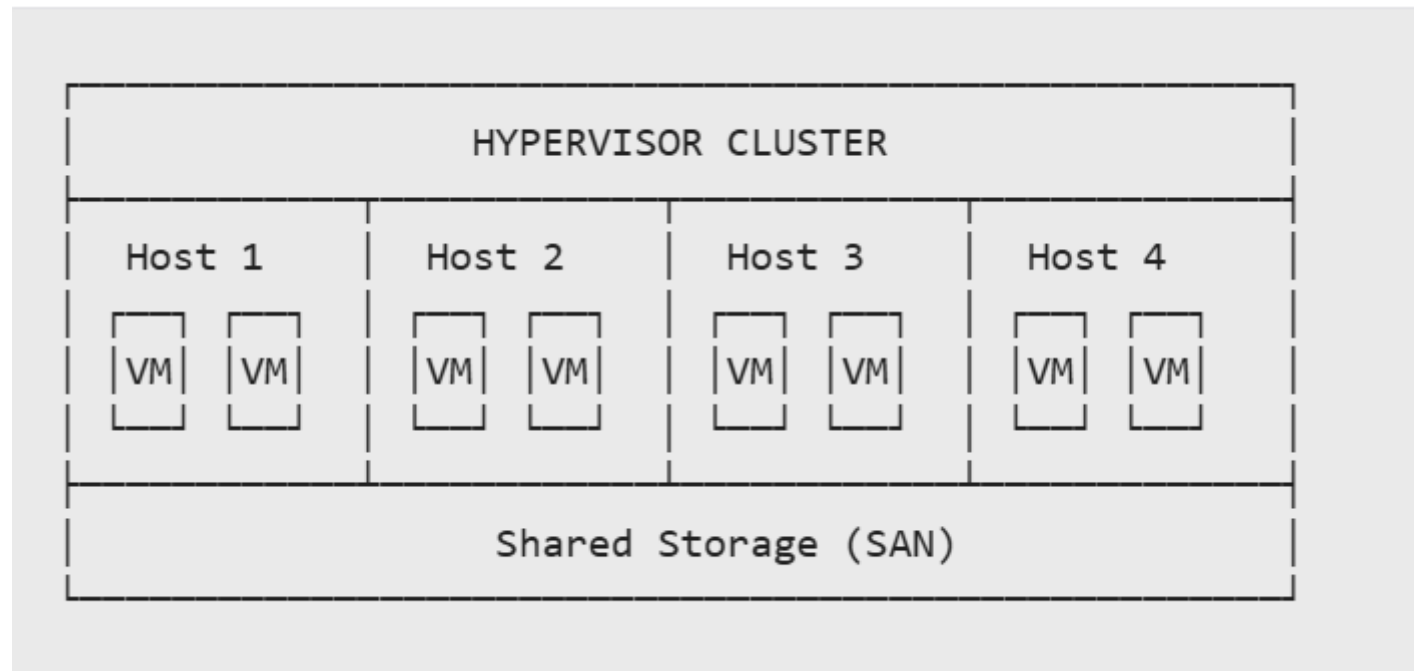
- **Terraform**: Infrastructure as Code across clouds
- **Kubernetes**: Container orchestration portability
- **Pulumi**: Multi-cloud IaC with programming languages

1.6 Hypervisor Clustering

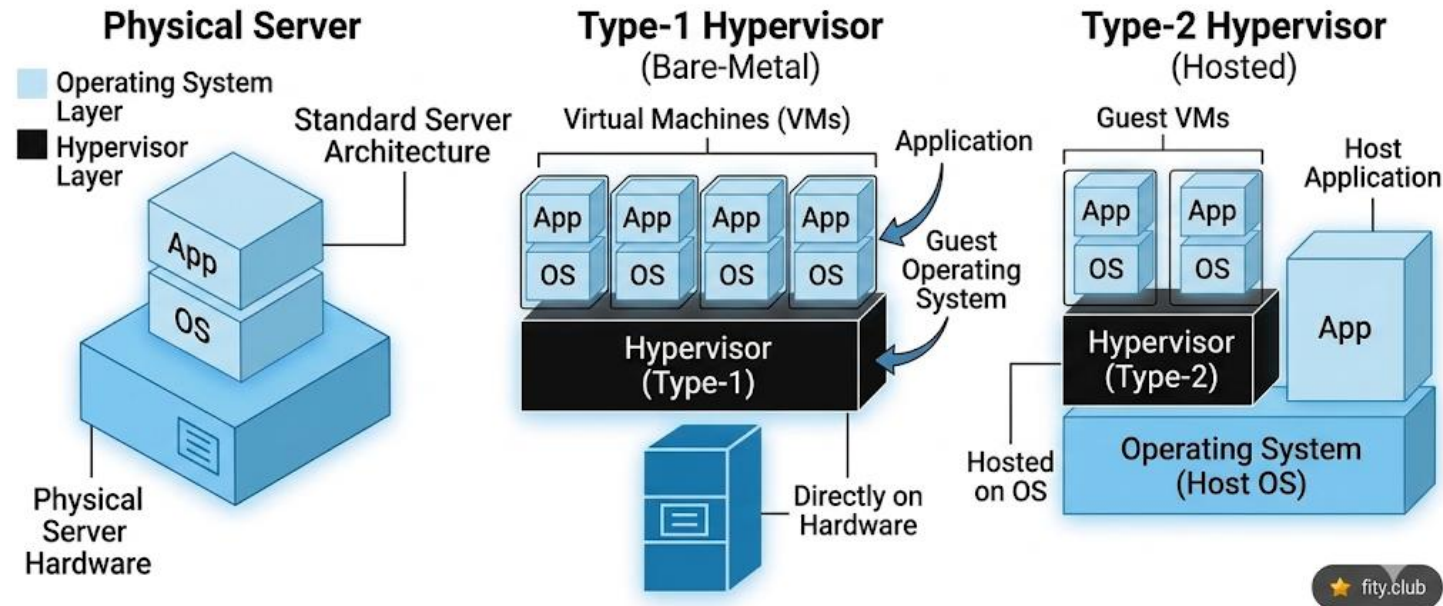
Concept

Multiple hypervisors work together to provide VM high availability. If one physical host fails, VMs automatically migrate to healthy hosts.

Hypervisor Architecture



1.6 Hypervisor Clustering



Hypervisor Types

Type 1 (Bare Metal)	Type 2 (Hosted)
Runs directly on hardware	Runs on host OS
VMware ESXi, Xen, Hyper-V	VirtualBox, VMware Workstation
Production use	Development/testing

AWS Implementation

- Nitro System:** AWS's custom hypervisor
- Placement Groups:** Control VM placement for HA
- Dedicated Hosts:** Full control over host placement

1.7 Cloud Balancing (DNS-Based Routing)

Concept

Intelligent traffic routing based on user location, server health, or custom policies using DNS.

Routing Policies

Policy	Description	Use Case
Simple	Single resource	Basic setup
Weighted	Percentage-based split	A/B testing, gradual migration
Latency	Lowest latency region	Global applications
Geolocation	User's geographic location	Regional content
Failover	Active-passive HA	Disaster recovery
Multivalue	Multiple healthy records	Load distribution

1.7 Cloud Balancing (DNS-Based Routing)

AWS Route 53 Configuration

```
# Create hosted zone
aws route53 create-hosted-zone \
  --name example.com \
  --caller-reference $(date +%s)

# Latency-based routing
aws route53 change-resource-record-sets \
  --hosted-zone-id Z123456 \
  --change-batch '{
    "Changes": [{
      "Action": "CREATE",
      "ResourceRecordSet": {
        "Name": "api.example.com",
        "Type": "A",
        "SetIdentifier": "us-east",
        "Region": "us-east-1",
        "TTL": 60,
        "ResourceRecords": [{"Value": "10.0.0.1"}]
      }
    }]
  }'
```

1.7 Cloud Balancing (DNS-Based Routing)

Global Architecture

```
User (India) → Route 53 → Mumbai Region (20ms)
User (USA)   → Route 53 → Virginia Region (15ms)
User (EU)    → Route 53 → Frankfurt Region (25ms)
```

Simple Example: Swiggy/Zomato Order

You order biryani on Swiggy:

- You're in Koramangala → order goes to nearest Koramangala restaurant
- Your friend in Indiranagar → order goes to Indiranagar outlet
- NOT routed to a single restaurant in Whitefield for all of Bangalore!

Route 53 does the same – routes users to nearest server.

1.8 Edge Computing

Concept

Process data at the network edge (near data source) instead of centralized cloud, reducing latency for time-sensitive applications.

Real-World Analogy: ATM vs Bank Branch

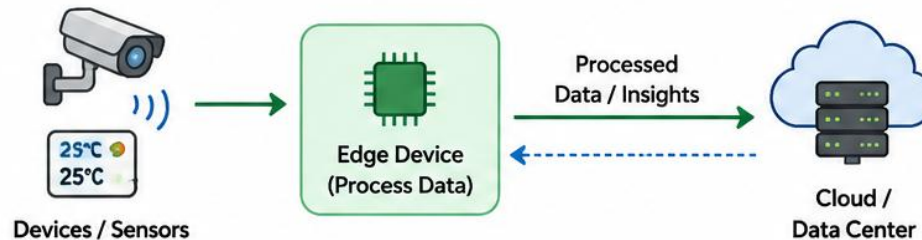
Withdraw ₹500:

- Bank Branch (Cloud): Drive 5km, wait in queue, 30 min → HIGH LATENCY
- ATM nearby (Edge): Walk 100m, instant cash → LOW LATENCY

Edge = Mini data centers close to users

Edge processing

Data is processed near the source of generation (at the “edge”) instead of sending it to a central server or cloud.



Benefits

- ✓ Low latency
- ✓ Saves bandwidth
- ✓ Works offline too
- ✓ Better privacy

1.8 Edge Computing

Application Examples

Application	Why Edge is Critical
Self-driving cars	Can't wait 200ms for "STOP" decision → edge AI in car
PUBG/Valorant	10ms vs 100ms = win vs lose → edge game servers
Factory robots	Emergency stop must be instant → local processing
Netflix	Videos cached at edge (ISP level) → no buffering

When Latency Matters

Application	Required Latency	Solution
Autonomous vehicles	< 10ms	Edge processing
AR/VR gaming	< 20ms	Edge + 5G
Industrial IoT	< 50ms	Fog computing
Video streaming	< 100ms	CDN caching

Edge vs Fog vs Cloud Processing

Where data is processed makes all the difference

CLOUD PROCESSING

Device → Cloud → Device
(100-200ms round trip)



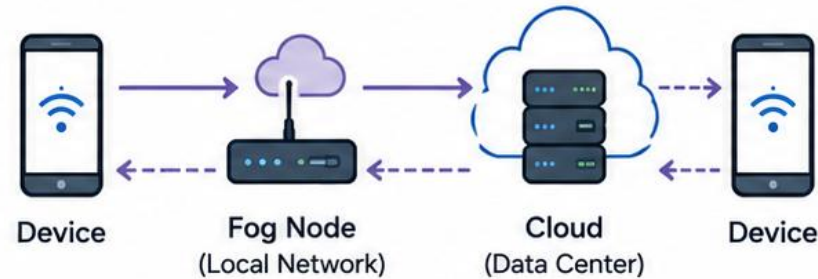
100-200ms
Round Trip Latency



- Data is sent to distant cloud
- Processed in data centers
- Higher latency (100-200ms)
- Depends on internet connectivity

FOG COMPUTING

Device → Fog → Cloud → Device
(10-50ms round trip)



10-50ms
Round Trip Latency



- Data processed on nearby fog nodes
- Intermediate layer between edge and cloud
- Medium latency (10-50ms)
- Reduces bandwidth and cloud load
- Better performance for many applications

EDGE PROCESSING

Device → Edge → Device
(<10ms processing)



<10ms
Processing Latency

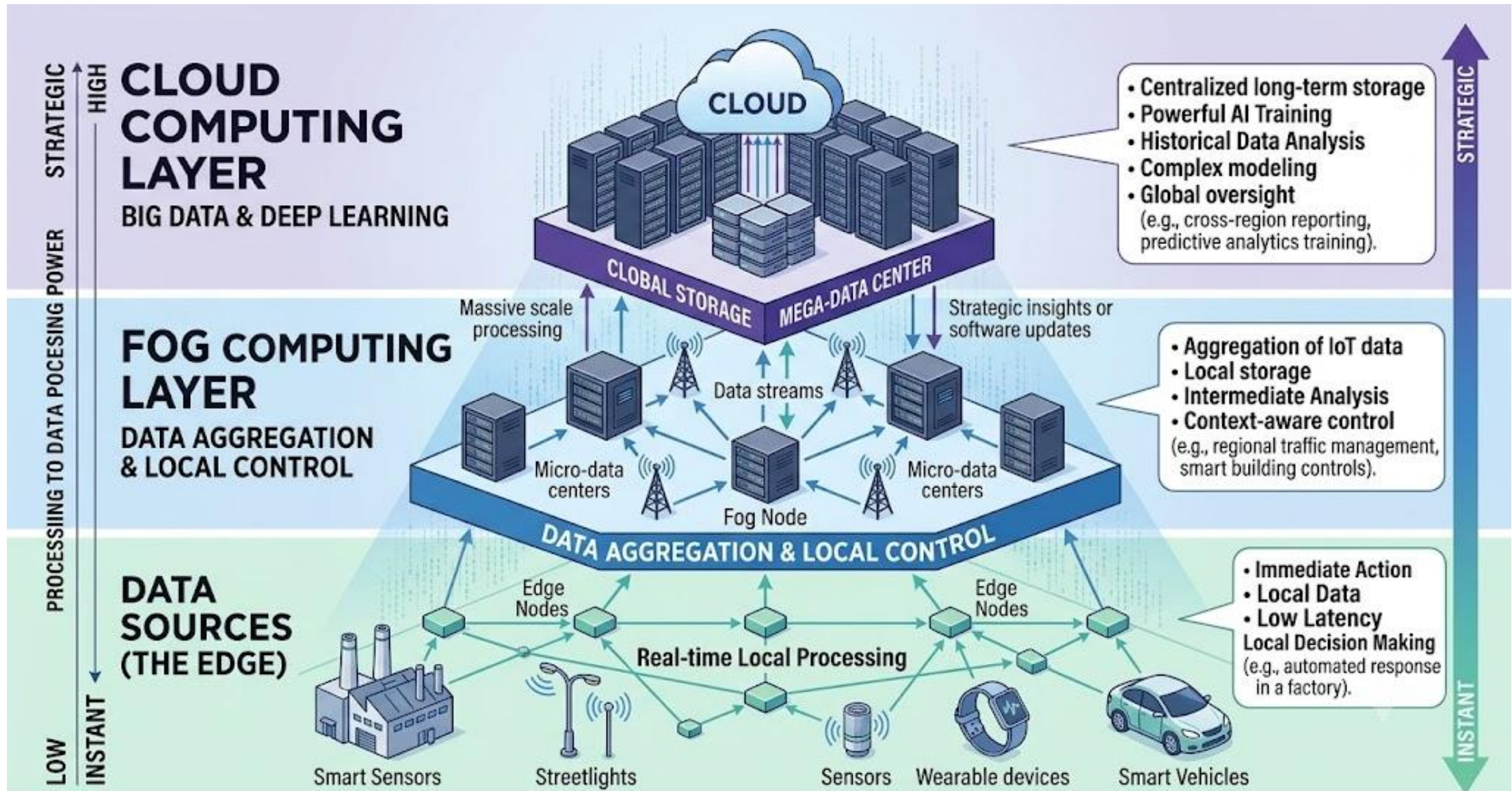


- Data processed near the source
- No need to go to cloud
- Very low latency (<10ms)
- Works even with poor/no internet
- Ideal for real-time applications



Summary: **Cloud** = Powerful but far | **Fog** = Smarter middle layer | **Edge** = Fastest and closest

1.8 Edge Computing



1.8 Edge Computing

AWS Edge Services

Service	Purpose
CloudFront	CDN for static content
Lambda@Edge	Run code at edge locations
Outposts	AWS infrastructure on-premises
Wavelength	AWS at 5G carrier edge
Snow Family	Portable edge computing

Lambda@Edge Example

```
// Modify response headers at edge
exports.handler = async (event) => {
  // Get the response from CloudFront
  const response = event.Records[0].cf.response;
  const headers = response.headers;

  // Add HSTS header - tells browsers "always use HTTPS"
  headers['strict-transport-security'] = [{
    key: 'Strict-Transport-Security',
    value: 'max-age=31536000; includeSubdomains'
  }];

  return response;
};
```

This Lambda@Edge code adds a **security header** to HTTP responses at the edge (CloudFront locations worldwide).

Think of it like Jio/Airtel towers across India:

- Without Lambda@Edge: Every call goes to Mumbai HQ for processing
- With Lambda@Edge: Call handled by nearest tower (Bangalore, Chennai, Delhi)

This code runs at 400+ CloudFront locations worldwide!

1.9 Fog Computing

Concept

Intermediate layer between edge devices and cloud. Provides more processing power than edge devices but lower latency than cloud.

Real-World Analogy: Regional Manager

Company structure:

- CEO (Cloud) → Makes big decisions, slow response
- Regional Manager (Fog) → Handles regional issues quickly
- Store Staff (Edge) → Handles immediate customer needs

Not everything needs CEO approval!

Fog computing

Adds an intermediate layer (fog nodes) between edge devices and the cloud. Fog nodes aggregate, process, and filter data before sending it up.



Benefits

- ✓ Reduces cloud load
- ✓ Improves efficiency
- ✓ Better scalability
- ✓ Supports mobility

1.9 Fog Computing

Application Examples

Industry	Fog Computing Use
Smart City Traffic	Local fog nodes adjust signals, cloud does city-wide planning
Hospital ICU	Fog aggregates patient vitals, alerts locally, sends trends to cloud
Oil Rig Sensors	Fog processes 10,000 sensor readings, sends only anomalies to cloud
Smart Warehouse	Fog coordinates robots locally, cloud manages inventory

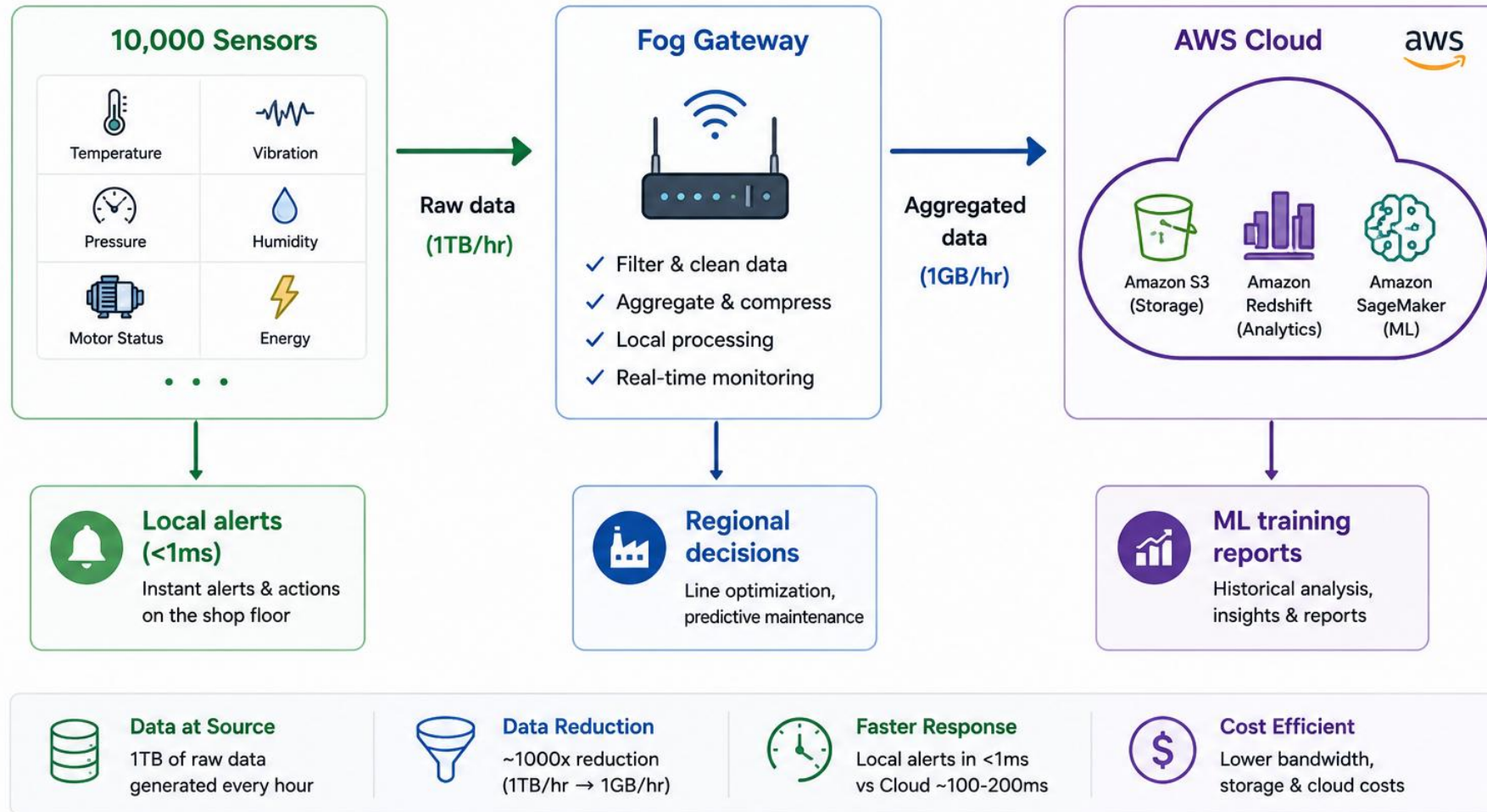
AWS IoT Greengrass

- Run Lambda functions on local devices
- Local messaging between devices
- Sync with cloud when connected
- Offline operation capability

CLOUD (AWS Region)	Unlimited compute, 50-200ms
FOG (Local Gateway)	Moderate compute, 1-10ms
EDGE (Device)	Limited compute, <1ms

1.9 Fog Computing

Use Case: Smart Factory



Outcome: Real-time operations, reduced downtime, lower costs, and data-driven decisions

1.10 Metacloud & Federated Cloud

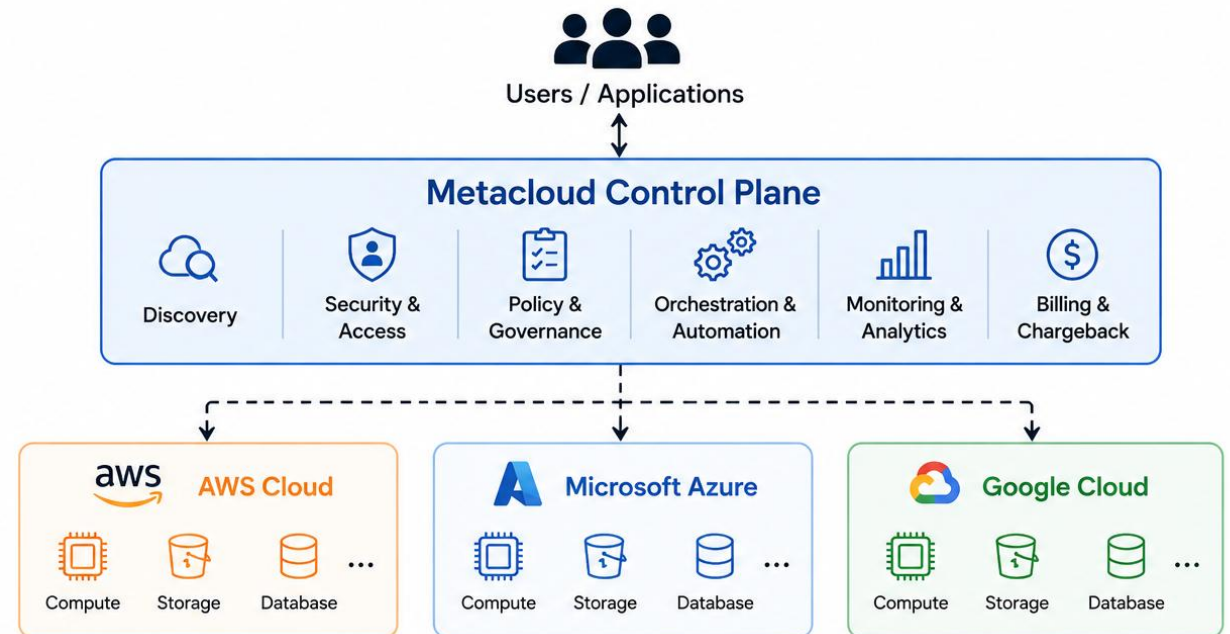
Metacloud

Single management plane for multiple clouds.

Tools:

- Kubernetes:** Container orchestration across clouds
- Terraform:** Unified IaC for all providers
- Crossplane:** Kubernetes-native cloud management

One control plane to manage and use multiple clouds as a single unified cloud.



Benefits



Avoid
Vendor Lock-in



Optimize
Cost



Improve
Performance



Increase
Resilience



Portability &
Flexibility

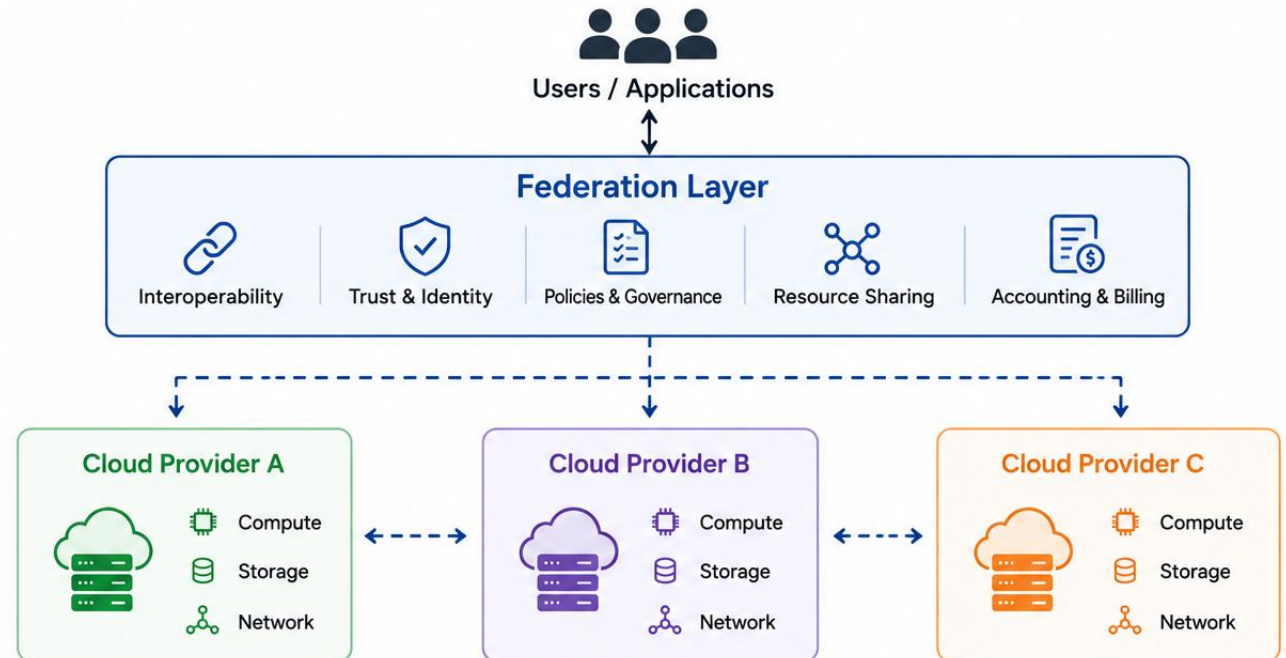
1.10 Metacloud & Federated Cloud

Federated Cloud

Independent organizations share resources while maintaining control.

Characteristics:

- Each organization keeps data sovereignty
- Shared compute resources via agreements
- Common identity federation (SSO)
- Example: European Grid Infrastructure (EGI)



Hands on Session: EC2 and S3 Buckets in AWS

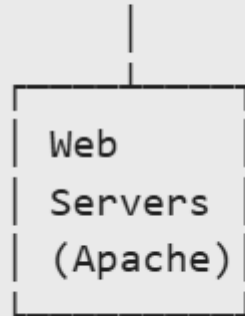
1.11 Lab Exercise: Deploy Scalable Web App

Objective

Deploy a load-balanced, auto-scaling web application on AWS.

Architecture

Internet → ALB → Auto Scaling Group (2-4 instances)



1.11 Lab Exercise: Deploy Scalable Web App

Step 1: Create Security Group

1. **Navigate:** EC2 Console → Security Groups → Create Security Group

2. **Configure:**

- Name: `web-server-sg`
- Description: "Allow HTTP from internet"
- VPC: Default VPC

3. **Add Inbound Rules:**

Type	Port	Source	Purpose
HTTP	80	0.0.0.0/0	Web traffic
SSH	22	My IP	Admin access (optional)

4. Click **Create Security Group**

Checkpoint: Note down the Security Group ID (sg-xxxxx)

1.11 Lab Exercise: Deploy Scalable Web App

Step 2: Create Launch Template

1. **Navigate:** EC2 → Launch Templates → Create Launch Template

2. **Configure:**

- Name: `web-server-template`
- AMI: Amazon Linux 2023 (Free Tier eligible)
- Instance type: `t2.micro` (Free Tier)
- Key pair: Select existing or create new
- Security Group: Select `web-server-sg`

3. **Advanced Details** → **User Data:** (paste this script)

4. Click **Create Launch Template**

Checkpoint: Launch Template created successfully

```
#!/bin/bash
yum update -y
yum install -y httpd stress
systemctl start httpd
systemctl enable httpd

# Create webpage showing server hostname
cat <<EOF > /var/www/html/index.html
<!DOCTYPE html>
<html>
<head><title>Cloud Lab</title></head>
<body style="font-family: Arial; text-align: center; padding: 50px;">
  <h1>Hello from AWS!</h1>
  <h2>Server: $(hostname)</h2>
  <p>Instance ID: $(curl -s http://169.254.169.254/latest/meta-data/instance-id)</p>
  <p>Availability Zone: $(curl -s http://169.254.169.254/latest/meta-data/placement/availability-zone)</p>
</body>
</html>
EOF
```

1.11 Lab Exercise: Deploy Scalable Web App

Step 2: Create Launch Template

1. **Navigate:** EC2 → Launch Templates → Create Launch Template

2. **Configure:**

- Name: `web-server-template`
- AMI: Amazon Linux 2023 (Free Tier eligible)
- Instance type: `t2.micro` (Free Tier)
- Key pair: Select existing or create new
- Security Group: Select `web-server-sg`

3. **Advanced Details** → **User Data:** (paste this script)

4. Click **Create Launch Template**

Checkpoint: Launch Template created successfully

```
#!/bin/bash
yum update -y
yum install -y httpd stress
systemctl start httpd
systemctl enable httpd

# Create webpage showing server hostname
cat <<EOF > /var/www/html/index.html
<!DOCTYPE html>
<html>
<head><title>Cloud Lab</title></head>
<body style="font-family: Arial; text-align: center; padding: 50px;">
  <h1>Hello from AWS!</h1>
  <h2>Server: $(hostname)</h2>
  <p>Instance ID: $(curl -s http://169.254.169.254/latest/meta-data/instance-id)</p>
  <p>Availability Zone: $(curl -s http://169.254.169.254/latest/meta-data/placement/availability-zone)</p>
</body>
</html>
EOF
```

1.11 Lab Exercise: Deploy Scalable Web App

Step 3: Create Target Group

1. **Navigate:** EC2 → Target Groups → Create Target Group

2. **Configure:**

- Target type: Instances
- Name: web-targets
- Protocol: HTTP, Port: 80
- VPC: Default VPC
- Health check path: /

3. Click **Next** → **Create Target Group** (don't register targets manually)

Checkpoint: Target Group ARN noted

1.11 Lab Exercise: Deploy Scalable Web App

Step 4: Create Application Load Balancer

1. **Navigate:** EC2 → Load Balancers → Create Load Balancer
2. **Select:** Application Load Balancer
3. **Configure:**
 - Name: `web-alb`
 - Scheme: Internet-facing
 - IP type: IPv4
 - VPC: Default VPC
 - Mappings: Select at least 2 Availability Zones (ap-south-1a, ap-south-1b)
4. **Security Group:** Create new or select existing allowing HTTP:80
5. **Listeners:**
 - HTTP:80 → Forward to `web-targets`
6. Click **Create Load Balancer**

Checkpoint: Note ALB DNS name (web-alb-xxxxx.ap-south-1.elb.amazonaws.com)

1.11 Lab Exercise: Deploy Scalable Web App

Step 5: Create Auto Scaling Group

1. **Navigate:** EC2 → Auto Scaling Groups → Create Auto Scaling Group

2. **Step 1 - Choose template:**

- Name: web-asg
- Launch Template: web-server-template

3. **Step 2 - Choose instance launch options:**

- VPC: Default VPC
- Subnets: Select 2+ subnets (different AZs)

4. **Step 3 - Configure advanced options:**

- Load balancing: Attach to existing load balancer (check the box)
- Choose: web-targets target group
- Health checks: Enable ELB health checks

5. **Step 4 - Configure group size:**

- Desired: 2
- Minimum: 2
- Maximum: 4 (keep low for Free Tier)

6. **Step 5 - Configure scaling policies:**

- Select: Target tracking scaling policy
- Metric: Average CPU Utilization
- Target value: 50 (lower for demo)

7. **Review and Create**

Checkpoint: ASG shows 2 instances launching

1.11 Lab Exercise: Deploy Scalable Web App

Step 5: Create Auto Scaling Group

1. **Navigate:** EC2 → Auto Scaling Groups → Create Auto Scaling Group

2. **Step 1 - Choose template:**

- Name: web-asg
- Launch Template: web-server-template

3. **Step 2 - Choose instance launch options:**

- VPC: Default VPC
- Subnets: Select 2+ subnets (different AZs)

4. **Step 3 - Configure advanced options:**

- Load balancing: Attach to existing load balancer (check the box)
- Choose: web-targets target group
- Health checks: Enable ELB health checks

5. **Step 4 - Configure group size:**

- Desired: 2
- Minimum: 2
- Maximum: 4 (keep low for Free Tier)

6. **Step 5 - Configure scaling policies:**

- Select: Target tracking scaling policy
- Metric: Average CPU Utilization
- Target value: 50 (lower for demo)

7. **Review and Create**

Checkpoint: ASG shows 2 instances launching

1.11 Lab Exercise: Deploy Scalable Web App

Step 6: Test Your Setup

6.1 Verify Load Balancing:

1. Wait 2-3 minutes for instances to pass health checks
2. Open ALB DNS in browser: `http://web-alb-xxxxx.ap-south-1.elb.amazonaws.com`
3. Refresh multiple times — observe different hostnames (load balancing working!)

6.2 Test Auto Scaling (Optional - generates CPU load):

SSH into one instance and run:

```
# Generate CPU stress for 2 minutes
stress --cpu 4 --timeout 120
```

Or use CloudWatch to observe:

1. Navigate: CloudWatch → Alarms
2. Watch for scaling alarm trigger
3. Check EC2 console for new instances launching

Success Criteria:

- ☐ ALB DNS loads webpage
- ☐ Different server names on refresh
- ☐ Instances show "Healthy" in Target Group

1.11 Lab Exercise: Deploy Scalable Web App

Step 7: Cleanup (IMPORTANT - Avoid charges!)

Delete in this order:

Step	Resource	Navigation
1	Auto Scaling Group	EC2 → Auto Scaling Groups → Delete
2	Load Balancer	EC2 → Load Balancers → Delete
3	Target Group	EC2 → Target Groups → Delete
4	Launch Template	EC2 → Launch Templates → Delete
5	Security Group	EC2 → Security Groups → Delete <code>web-server-sg</code>

CLI Cleanup (alternative):

```
# Delete ASG (terminates instances automatically)
aws autoscaling delete-auto-scaling-group \
  --auto-scaling-group-name web-asg \
  --force-delete

# Wait for instances to terminate, then delete ALB
aws elbv2 delete-load-balancer \
  --load-balancer-arn arn:aws:elasticloadbalancing:ap-south-1:...

# Delete Target Group
aws elbv2 delete-target-group \
  --target-group-arn arn:aws:elasticloadbalancing:ap-south-1:...
```


1.11 Lab Exercise: Deploy Scalable Web App

Troubleshooting

Problem	Solution
ALB shows "503 Service Unavailable"	Wait 2-3 min for health checks; verify Security Group allows HTTP
Instances stuck in "Unhealthy"	Check User Data script; SSH and verify Apache running (<code>systemctl status httpd</code>)
Can't access ALB DNS	Check ALB Security Group allows inbound HTTP:80 from 0.0.0.0/0
Auto Scaling not triggering	Lower target CPU to 30%; generate more load

1.12 Lab Exercise: S3 Static Website Hosting

Objective

Deploy a static website on S3 without managing any servers — auto-scales to millions of visitors.

Real-World Analogy: Google Drive vs Renting a Server Room

Hosting your portfolio website:

Traditional Way (EC2):

- Rent a server room (EC2 instance)
- Pay electricity 24/7 even with 0 visitors
- Manage security, updates, scaling yourself
- Cost: ₹500-2000/month minimum

S3 Way:

- Like putting files on Google Drive and sharing the link
- Pay only for storage + actual visitors
- AWS handles security, scaling, uptime
- Cost: ₹2-5/month for small sites!

S3 Static Hosting = Google Drive that auto-scales to Netflix traffic levels

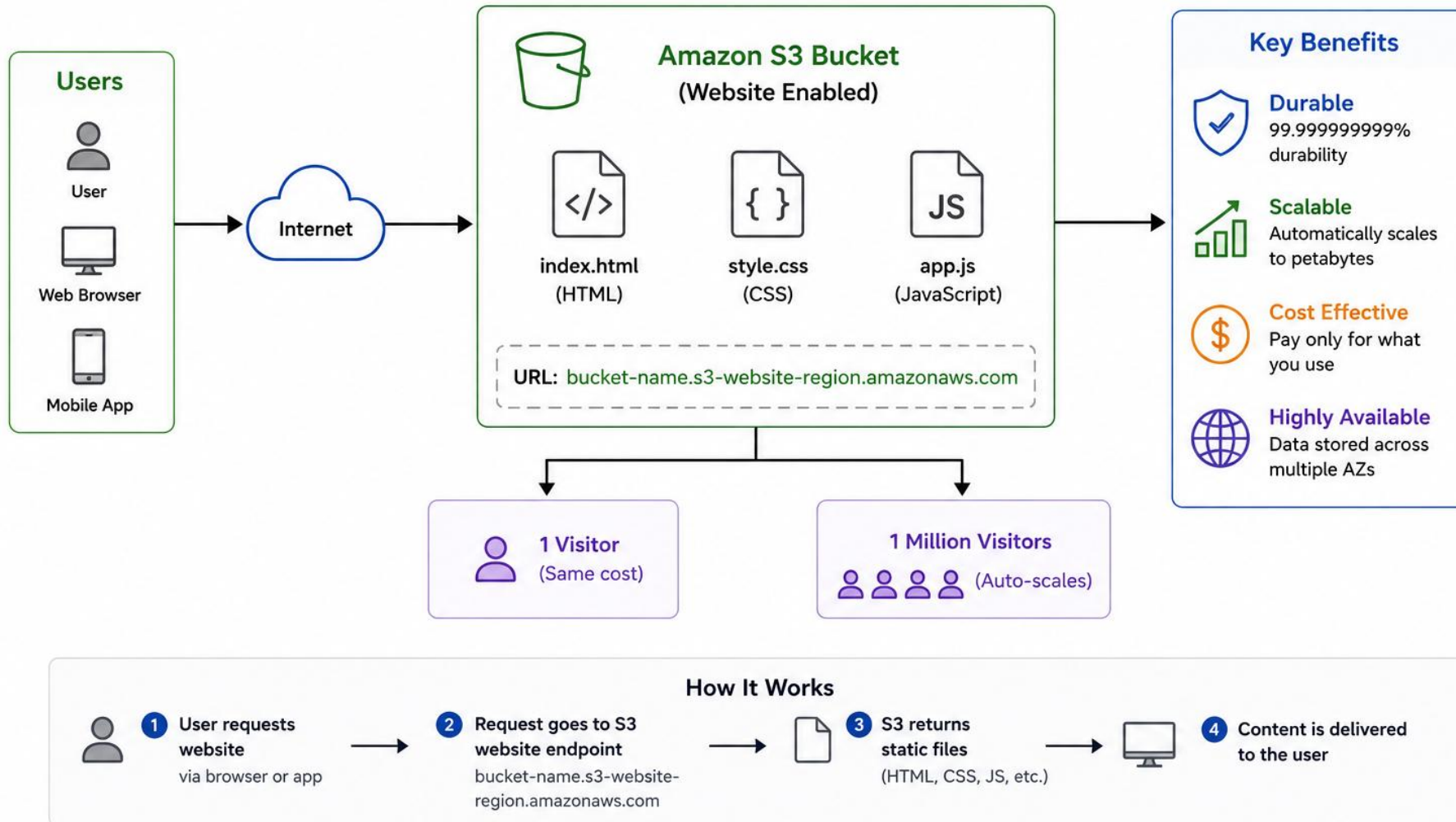
1.12 Lab Exercise: S3 Static Website Hosting

Application Examples

Website Type	Why S3 is Perfect
Zerodha Varsity	Documentation site — millions of readers, zero server management
Razorpay Docs	API documentation — scales during developer conferences
Portfolio sites	Students hosting resumes — costs ₹0-5/month
React/Vue SPAs	Single Page Apps — just HTML/CSS/JS, no backend needed
Marketing landing pages	Product launches — handles viral traffic spikes

1.12 Lab Exercise: S3 Static Website Hosting

Architecture



1.12 Lab Exercise: S3 Static Website Hosting

Step 1: Create S3 Bucket

1. **Navigate:** AWS Console → S3 → Create Bucket

2. **Configure:**

- Bucket name: `your-name-portfolio-2024` (must be globally unique!)
- Region: Asia Pacific (Mumbai) ap-south-1
- **Uncheck** "Block all public access" (required for website)
- Acknowledge the warning checkbox

3. Click **Create Bucket**

Why globally unique? S3 bucket names are like domain names — only one `amazon-website` can exist worldwide.

Checkpoint: Bucket appears in your S3 bucket list

1.12 Lab Exercise: S3 Static Website Hosting

Step 2: Enable Static Website Hosting

1. **Navigate:** Click your bucket → Properties tab
2. **Scroll to:** Static website hosting → Edit
3. **Configure:**
 - Static website hosting: **Enable**
 - Hosting type: Host a static website
 - Index document: `index.html`
 - Error document: `error.html` (optional)
4. Click **Save changes**

Checkpoint: Note the **Bucket website endpoint** URL (you'll need this!)

```
http://your-name-portfolio-2024.s3-website.ap-south-1.amazonaws.com
```

Step 3: Create and Upload Website Files

3.1 Create index.html locally:

```
<!DOCTYPE html>
<html>
<head>
  <title>My Cloud Portfolio</title>
  <style>
    body {
      font-family: Arial;
      text-align: center;
      padding: 50px;
      background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
      color: white;
      min-height: 100vh;
      margin: 0;
    }
    .container {
      background: rgba(255,255,255,0.1);
      padding: 40px;
      border-radius: 20px;
      max-width: 600px;
      margin: 0 auto;
    }
    h1 { font-size: 2.5em; margin-bottom: 10px; }
    p { font-size: 1.2em; opacity: 0.9; }
    .badge {
      background: #00d4aa;
      padding: 5px 15px;
      border-radius: 20px;
      font-size: 0.9em;
    }
  </style>
</head>
<body>
  <div class="container">
    <h1>Hello from S3!</h1>
    <p>This website is hosted on AWS S3</p>
    <p>No servers. No maintenance. Auto-scales to millions.</p>
    <span class="badge">Cost: ~₹2/month</span>
  </div>
</body>
</html>
```

1.12 Lab Exercise: S3 Static Website Hosting

3.2 Upload via Console:

1. **Navigate:** Your bucket → Objects tab → Upload
2. **Add files:** Select your `index.html`
3. Click **Upload**

CLI Alternative:

```
# Upload single file
aws s3 cp index.html s3://your-name-portfolio-2024/

# Upload entire folder
aws s3 sync ./my-website s3://your-name-portfolio-2024/
```

What `s3 sync` does: Like rsync — only uploads changed files, deletes removed files. Perfect for deployments.

Checkpoint: `index.html` visible in bucket Objects list

1.12 Lab Exercise: S3 Static Website Hosting

Step 4: Configure Bucket Policy (Make Public)

1. **Navigate:** Your bucket → Permissions tab → Bucket policy → Edit
2. **Paste this policy:**

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicReadGetObject",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3::your-name-portfolio-2024/*"
    }
  ]
}
```

3. **Replace** `your-name-portfolio-2024` with your actual bucket name
4. Click **Save changes**

Policy Explained:

"Version": "2012-10-17"	→ Policy language version (always use this)
"Sid": "PublicReadGetObject"	→ Human-readable ID for this rule
"Effect": "Allow"	→ Permission granted (not denied)
"Principal": "*"	→ Who can access? Everyone (*)
"Action": "s3:GetObject"	→ What can they do? Only READ files
"Resource": ".../*"	→ Which files? All files in bucket

Security Note: This only allows READ. Visitors cannot upload, delete, or list your files.

Checkpoint: "Publicly accessible" badge appears on bucket

1.12 Lab Exercise: S3 Static Website Hosting

Step 5: Test Your Website

1. **Navigate:** Bucket → Properties → Static website hosting
2. **Copy:** Bucket website endpoint URL
3. **Open in browser:** `http://your-name-portfolio-2024.s3-website.ap-south-1.amazonaws.com`

Success Criteria:

- ☐ Website loads without errors
- ☐ Styling displays correctly
- ☐ URL shows S3 website endpoint format

1.12 Lab Exercise: S3 Static Website Hosting

Step 6: Cleanup (Optional - Keep if you want the site live)

Delete bucket contents and bucket:

Step	Action	Navigation
1	Empty bucket	S3 → Bucket → Empty → Confirm
2	Delete bucket	S3 → Bucket → Delete → Type name → Confirm

CLI Cleanup:

```
# Delete all objects (required before bucket deletion)
aws s3 rm s3://your-name-portfolio-2024 --recursive

# Delete bucket
aws s3 rb s3://your-name-portfolio-2024
```

What `--recursive` **does:** S3 buckets must be empty before deletion. This flag deletes all files inside first.

1.12 Lab Exercise: S3 Static Website Hosting

Troubleshooting

Problem	Cause	Solution
403 Forbidden	Bucket policy missing/wrong	Check Step 4; verify bucket name in policy matches exactly
404 Not Found	index.html missing or wrong name	File must be named exactly <code>index.html</code> (case-sensitive)
"Access Denied" when saving policy	Block Public Access enabled	Bucket → Permissions → Block public access → Edit → Uncheck all
Website URL not working	Using wrong URL format	Use the S3 website endpoint, NOT the object URL
CSS/Images not loading	Files not uploaded or wrong path	Check all files uploaded; use relative paths in HTML

URL Formats (Common Confusion!)

URL Type	Format	Use For
Website endpoint	<code>bucket.s3-website-region.amazonaws.com</code>	Browsing website
Object URL	<code>bucket.s3.region.amazonaws.com/file.html</code>	Direct file access (won't render HTML)

1.12 Lab Exercise: S3 Static Website Hosting

Production Upgrade: Add CloudFront CDN

For real production websites, add CloudFront in front of S3:

Benefits:

- HTTPS support (required for modern browsers)
- Custom domain (yourname.com instead of S3 URL)
- Global edge caching (fast worldwide)
- DDoS protection (AWS Shield)
- Cost: ~\$0.085/GB data transfer

Architecture with CloudFront:

Users → CloudFront (400+ edge locations) → S3 Bucket
↓
HTTPS, caching, custom domain

1.12 Lab Exercise: S3 Static Website Hosting

What You Learned

- Created S3 bucket with globally unique name
- Enabled static website hosting feature
- Configured bucket policy for public read access
- Understood the difference between S3 URLs and website endpoints
- Deployed a website with zero server management
- Learned when to upgrade to CloudFront for production

Architecture Pattern

Pattern / Architecture	Use When...
Workload Distribution	High traffic needs reliability
Elastic Capacity	Variable / unpredictable traffic
Multi-Cloud	Need redundancy; avoid vendor lock-in
Hypervisor Cluster	Need high availability / live migration
Cloud Balancing	Global users need low latency everywhere
Edge Computing	Real-time decisions; processing close to users
Fog Computing	IoT with regional processing capability
Metacloud	Managing multiple clouds from one place
Federated Cloud	Organizations sharing resources while staying independent

Cloud Resources

Pattern / Architecture	Cloud Solution
Workload Distribution	Use Load Balancers (ELB)
Elastic Capacity	Use Auto Scaling Groups
Multi-Cloud	Avoid vendor lock-in
Hypervisor Cluster	High Availability
Cloud Balancing	Route 53 for smart routing
Edge Computing	Process near the data (CloudFront)
Fog Computing	Middle layer (IoT Greengrass)
Metacloud	Kubernetes for multi-cloud
Federated Cloud	Independent clouds cooperating

AWS Services at Glance

AWS Service	One-Line Description
EC2	Virtual servers in the cloud
S3	Object storage for files, images, and backups
RDS	Managed relational databases like MySQL & PostgreSQL
VPC	Private networking environment in AWS
IAM	Identity and access management for users and permissions
Security Groups	Virtual firewalls controlling inbound/outbound traffic
Route 53	DNS management and traffic routing service
CloudFront	Content Delivery Network (CDN) for faster access

AWS Services at Glance

AWS Service	One-Line Description
ELB	Load Balancer to distribute application traffic
Auto Scaling	Automatically scale servers based on demand
CloudTrail	Monitor and log AWS account activities
GuardDuty	Intelligent threat detection and security monitoring
Macie	Detect and protect sensitive data in S3
KMS	Encryption key creation and management service
EKS	Managed Kubernetes service on AWS
IoT Greengrass	Run AWS services on edge and fog devices



THANK YOU!

Good luck on your cloud journey